do-while Loop in Java

The `do-while` loop is similar to the `while` loop but with a key difference: in a `do-while` loop, the condition is checked after the block of code is executed, meaning the loop will always execute **at least once**. This is useful when you want to ensure that the code runs at least once, regardless of the condition.

Syntax:

```
do {
    // Code to be executed
} while (condition);
```

- `condition`: A boolean expression that is evaluated after each iteration. If the condition is `true`, the loop executes again.
- The code inside the `{}` block runs first, then the condition is checked. If the condition is `true`, the loop continues. If the condition is `false`, the loop terminates.

How It Works:

1. The code inside the `do` block is executed **once**, regardless of the condition.
2. After the code executes, the condition is evaluated.
3. If the condition is `true`, the loop repeats.
4. If the condition is `false`, the loop stops.

Example 1: Basic `do-while` Loop

```
public class Main {
    public static void main(String[] args) {
        int count = 1;

        do {
            System.out.println("Count: " + count);
            count++;  // Increment count by 1
        } while (count <= 5);
    }
}
```

Explanation:

- The loop starts by executing the code inside the `do` block (printing "Count: 1").
- The value of `count` is incremented after each iteration.
- After printing the count, the condition `count <= 5` is checked.
- The loop continues until `count` becomes `6`, at which point the condition becomes `false`, and the loop terminates.

Output:

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

Example 2: `do-while` Loop with Condition False Initially

```java
public class Main {
    public static void main(String[] args) {
        int number = 6;

        do {
            System.out.println("Number: " + number);
            number++;  // Increment number by 1
        } while (number <= 5);
    }
}
```

Explanation:

- Even though the condition is `false` initially (`number = 6`), the code inside the `do` block will be executed **once**.
- The `number` is printed first (which is `6`), and then incremented.
- After that, the condition `number <= 5` is checked. Since it is now `false`, the loop terminates.

Output:

```
Number: 6
```

Example 3: `do-while` Loop with `break` Statement

```
public class Main {
    public static void main(String[] args) {
        int number = 1;

        do {
            if (number == 4) {
                break;  // Exit the loop when number equals 4
            }
            System.out.println("Number: " + number);
            number++;  // Increment number by 1
        } while (number <= 5);
    }
}
```

Explanation:

- The loop will print the numbers from 1 to 3.
- When `number` becomes `4`, the `break` statement is executed, which immediately exits the loop.
- The loop will not execute for `number = 4` or `5`.

Output:

```
Number: 1
Number: 2
Number: 3
```

Example 4: `do-while` Loop with `continue` Statement

```
public class Main {
    public static void main(String[] args) {
        int number = 0;

        do {
            number++;
            if (number == 3) {
                continue;  // Skip this iteration when number is 3
            }
            System.out.println("Number: " + number);
```

```
        } while (number < 5);
    }
}
```

Explanation:

- The loop will print numbers from 1 to 5.
- When `number` equals `3`, the `continue` statement is executed, which skips the rest of the code inside the loop for that iteration, skipping the print statement for `3`.
- The loop continues with the next iteration.

Output:

```
Number: 1
Number: 2
Number: 4
Number: 5
```

Key Points:

1. **Guaranteed First Execution**: Unlike the `while` loop, the `do-while` loop executes the code block **at least once**, even if the condition is `false` initially.
2. **Condition Checking After Execution**: The condition is checked after the code block executes, allowing the loop to run at least one time.
3. **Control Statements**: You can use `break` to exit the loop early or `continue` to skip the current iteration and proceed with the next one.
4. **Infinite Loop**: If the condition is always `true`, the loop will run indefinitely unless you use a `break` statement or external interruption.
